

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE

SEMINAR REPORT

Object-Oriented Design Patterns

Course: Programming Systems — CS202
Class: 25A02
Group ID: 53
Seminar topics: Decorator, Facade, and Strategy
Institution: University of Science, VNU-HCM

Ho Chi Minh City, August 2026

Seminar Group Members

The seminar is prepared by Group 53, consisting of two members with complementary responsibilities.

Member 1

Name	Từ Hoàng Anh
Student ID	25125005
Primary role	Research and Implementation Lead
Responsibilities	Research pattern theory, develop and test C++ examples, verify class diagrams, and prepare technical demonstrations.
Contribution	50%

Member 2

Name	Nguyễn Phúc Khánh
Student ID	25125086
Primary role	Documentation and Presentation Lead
Responsibilities	Organize the report, design presentation slides, manage references, prepare the quiz, and coordinate the oral seminar.
Contribution	50%

Abstract

This report presents three Gang of Four object-oriented design patterns: *Decorator*, *Strategy*, and *Facade*. For each pattern, the report begins with a practical software-design problem, examines a naive implementation, introduces the pattern's intent and participants, and then discusses a C++-oriented implementation. The report also evaluates benefits, limitations, and real-world uses. Together, the patterns demonstrate three different uses of object composition: dynamically adding responsibilities, interchanging algorithms, and simplifying access to a complex subsystem.

Contents

Seminar Group Members	i
Abstract	ii
1 Introduction	1
1.1 Background	1
1.2 Objectives and scope	1
1.3 Method	1
2 Decorator Pattern	2
2.1 Real-world problem: customizing coffee beverages	2
2.1.1 Real-world analogy	2
2.2 Naive solution without Decorator	2
2.3 Problems in the naive design	3
2.4 Pattern intent and applicability	4
2.5 General class structure	4
2.6 Coffee-beverage class structure	5
2.7 Implementation with Decorator	5
2.7.1 Component and concrete beverages	5
2.7.2 Decorator base class and concrete condiments	6
2.7.3 Client and dynamic wrapping	7
2.8 Technical considerations	7
2.8.1 Ownership and memory management	7
2.8.2 Order of execution dependence	7
2.8.3 Interface transparency vs. type inspection	8
2.8.4 Performance and delegation overhead	8
2.8.5 Testing and mock decorators	8
2.9 Advantages and disadvantages	8
2.10 Other real-world applications	8

2.11	Relationship to similar patterns	10
2.12	Multiple Choice Quiz	10
3	Facade Pattern	11
3.1	Real-world problem: operating a home theater	11
3.2	Naive solution without Facade	11
3.3	Problems in the naive design	12
3.4	Pattern intent and applicability	12
3.5	General class structure	12
3.6	Home-theater class structure	13
3.7	Implementation with Facade	14
3.8	Technical considerations	15
3.8.1	Configuration and flexibility	15
3.8.2	Exception safety	15
3.8.3	Open/Closed Principle trade-off	16
3.8.4	Avoiding a god object	16
3.9	Advantages and disadvantages	16
3.10	Other real-world applications	16
3.11	Relationship to similar patterns	17
3.12	Multiple Choice Quiz	17
4	Strategy Pattern	18
4.1	Real-world problem: navigation route planning	18
4.1.1	Real-world analogy	18
4.2	Naive solution without Strategy	18
4.3	Problems in the naive design	19
4.4	Pattern intent and applicability	20
4.5	General class structure	20
4.6	Navigation class structure	21
4.7	Implementation with Strategy	21
4.7.1	Strategy interface and concrete algorithms	21
4.7.2	Context	22
4.7.3	Client and runtime switching	22
4.8	Technical considerations	23
4.8.1	Pointers, ownership, and object slicing	23
4.8.2	Const correctness and demonstration parameters	23
4.8.3	Client coupling and creation	23

4.8.4	Performance	23
4.8.5	Testing	24
4.9	Advantages and disadvantages	24
4.10	Other real-world applications	24
4.11	Relationship to similar patterns	25
4.12	Multiple Choice Quiz	25
5	Comparison and Combined Use	26
6	Conclusion	27
A	Submission Checklist	28
B	AI Usage Declaration	29
B.1	Declaration of AI use	29
B.2	AI Usage Notes	29
B.3	Student responsibility statement	29
	References	30

Chapter 1

Introduction

1.1 Background

Object-oriented programming provides encapsulation, inheritance, and polymorphism, but these mechanisms alone do not guarantee maintainable design. As systems grow, recurring problems appear: behavior must be extended without creating many subclasses, algorithms must change without rewriting a context, and clients must interact with complicated subsystems without depending on all their details. Design patterns record reusable structures for addressing such problems.

The Gang of Four catalogue groups patterns into three categories:

- **Creational patterns** control object creation.
- **Structural patterns** organize classes and objects.
- **Behavioral patterns** organize responsibilities and interaction.

Decorator and Facade are structural patterns, while Strategy is behavioral. The seminar emphasizes the principle *favor composition over inheritance* and relates each pattern to the SOLID principles.

1.2 Objectives and scope

The objectives of this seminar are to:

1. recognize the design problems that motivate each pattern;
2. identify the participants and relationships in each pattern;
3. compare naive and pattern-based C++ designs;
4. evaluate benefits, limitations, and appropriate use cases; and
5. distinguish patterns that can appear structurally similar.

The scope is limited to Decorator, Strategy, and Facade. The examples are educational implementations intended to make the object relationships clear; production systems would additionally require error handling, ownership rules, tests, and domain-specific behavior.

1.3 Method

Each pattern follows a problem-first layout: motivating example, naive solution, formal structure, C++ implementation, trade-offs, and applications.

Chapter 2

Decorator Pattern

2.1 Real-world problem: customizing coffee beverages

Point-of-sale (POS) systems for coffee shops must calculate prices and print descriptions for customized drinks. A beverage order begins with a base coffee and allows customers to add arbitrary combinations of condiments:

- base beverages include **Espresso**, **Dark Roast**, **House Blend**, and **Decaf**;
- condiments include **Steamed Milk**, **Mocha** (chocolate), **Soy**, and **Whipped Cream**;
- customers may request multiple servings of the same condiment (for example, double mocha); and
- condiment prices vary based on the specific add-on requested.

Constructing a beverage order requires a flexible workflow:

1. select a base beverage with a default description and base cost;
2. wrap the beverage with a requested condiment, extending its description and adding the condiment's price;
3. repeat condiment additions dynamically as requested by the customer; and
4. compute the final bill by summing up costs through the decorated structure.

Calculating the final cost dynamically without bloating the class hierarchy or hardcoding combinations into a base class is the core architectural challenge.

2.1.1 Real-world analogy

Consider a person dressing for extreme cold weather. A person starts with base attire (a t-shirt). As temperature drops, they put on a sweater over the shirt, add a winter coat over the sweater, and finally put on a waterproof raincoat over the coat. Each garment wraps the wearer (and all previous layers), adding distinct capabilities such as insulation or waterproofing. From the outside, the person remains a single wearable entity, but their behavior and characteristics are extended dynamically layer by layer without altering the person's fundamental body structure [3].

2.2 Naive solution without Decorator

A naive design relies purely on class inheritance, creating a distinct subclass for every possible combination of beverage and condiments:

```
1 // Naive approach 1: Subclass explosion
```



```

2  class Espresso { public: double cost() { return 1.99; } };
3  class EspressoWithMilk { public: double cost() { return 1.99 + 0.10; }
   };
4  class EspressoWithMilkAndMocha { public: double cost() { return 1.99 +
   0.10 + 0.20; } };
5  class DarkRoastWithSoyAndWhip { public: double cost() { return 0.99 +
   0.15 + 0.10; } };
6
7  // Naive approach 2: Fat base class with boolean flags
8  class Beverage {
9  protected:
10     std::string description;
11     bool milk = false;
12     bool mocha = false;
13     bool soy = false;
14     bool whip = false;
15 public:
16     virtual double cost() {
17         double total = 0.0;
18         if (hasMilk()) total += 0.10;
19         if (hasMocha()) total += 0.20;
20         if (hasSoy()) total += 0.15;
21         if (hasWhip()) total += 0.10;
22         return total;
23     }
24     bool hasMilk() const { return milk; }
25     void setMilk(bool m) { milk = m; }
26     // ... getter and setters for all other condiments ...
27 };
28
29 class DarkRoast : public Beverage {
30 public:
31     DarkRoast() { description = "Dark Roast Coffee"; }
32     double cost() override { return 0.99 + Beverage::cost(); }
33 };

```

While both approaches may compile and calculate costs, they introduce severe structural deficiencies as the system scales.

2.3 Problems in the naive design

1. **Class explosion.** Creating explicit subclasses for every beverage and condiment combination yields an exponential growth of classes ($O(2^n)$ for n condiments), creating a maintenance nightmare.
2. **Open/Closed Principle violation.** Adding a new condiment (e.g., Caramel) requires modifying the base `Beverage` class and updating its `cost()` logic, risking breakage across all existing drinks.
3. **Rigid compile-time binding.** Subclassing binds behavior statically at compile time. Clients cannot add or remove condiments dynamically during runtime based on user interaction.
4. **Inability to support multiple portions.** A boolean flag approach cannot cleanly represent a customer ordering double Mocha or triple Whip without adding extra integer counters to the base class.
5. **Inappropriate inheritance.** Base beverages should not carry state or knowledge of every potential condiment that might ever be invented.

2.4 Pattern intent and applicability

Decorator is a **structural design pattern**. Its intent is to attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality [1]. It is appropriate when:

- responsibilities need to be added to individual objects dynamically and transparently without affecting other objects;
- static inheritance is impractical due to class explosion or rigid hierarchies; or
- responsibilities can be withdrawn dynamically at runtime.

Decorator achieves transparent extension because the decorator shares the same abstract interface as the component it decorates (both *is-a* and *has-a*). The client remains unaware of whether it is interacting with a raw component or a heavily decorated chain of wrappers.

2.5 General class structure

Figure 2.1 shows the general structure of the Decorator pattern. The **Decorator** class implements the **Component** interface and maintains a reference to a **Component** instance, delegating requests to it while adding pre- or post-processing operations.

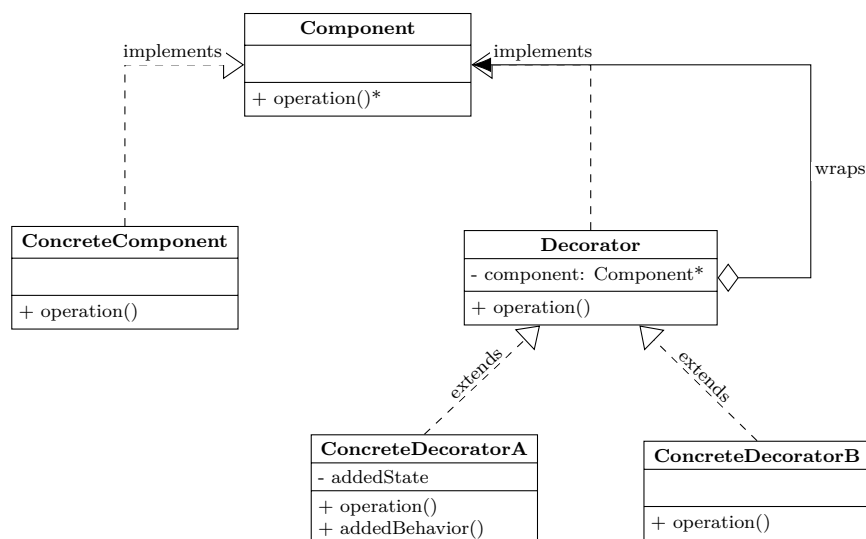


Figure 2.1: General Decorator structure showing transparent wrapping and interface realization.

The primary participants are:

- **Component**: defines the common interface for objects that can have responsibilities added to them dynamically;
- **ConcreteComponent**: defines an object to which additional responsibilities can be attached;
- **Decorator**: maintains a reference to a **Component** object and defines an interface that conforms to **Component**'s interface; and
- **ConcreteDecorator**: adds responsibilities to the wrapped component by performing additional operations before or after delegating to the enclosed object.

2.6 Coffee-beverage class structure

In Figure 2.2, `Beverage` serves as the abstract component. `Espresso` and `DarkRoast` are concrete components. `CondimentDecorator` inherits from `Beverage` and wraps an inner `Beverage` pointer. Concrete decorators like `Milk`, `Mocha`, and `Whip` override `getDescription()` and `cost()` to add their specific details and pricing.

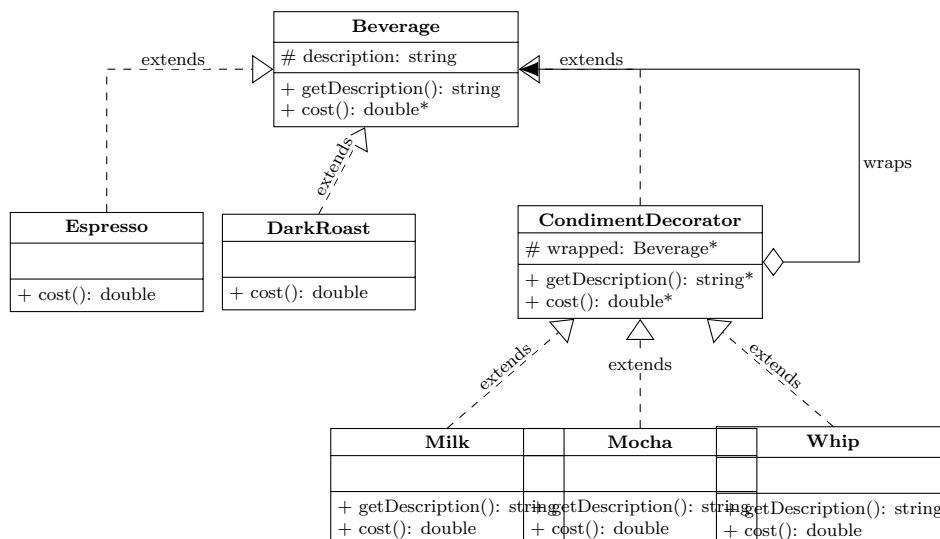


Figure 2.2: Decorator pattern applied to the coffee beverage customization system.

2.7 Implementation with Decorator

The standard C++11 implementation uses smart pointers (`std::unique_ptr`) to manage ownership cleanly along the object composition chain.

2.7.1 Component and concrete beverages

The abstract `Beverage` class defines the core interface, while concrete classes like `Espresso` and `DarkRoast` provide base implementations:

```

1 #include <iostream>
2 #include <memory>
3 #include <string>
4
5 // Abstract Component
6 class Beverage {
7 protected:
8     std::string description = "Unknown Beverage";
9 public:
10     virtual ~Beverage() = default;
11     virtual std::string getDescription() const { return description; }
12     virtual double cost() const = 0;
13 };
14
15 // Concrete Component: Espresso
16 class Espresso : public Beverage {
17 public:

```

```

18     Espresso() { description = "Espresso"; }
19     double cost() const override { return 1.99; }
20 };
21
22 // Concrete Component: Dark Roast
23 class DarkRoast : public Beverage {
24 public:
25     DarkRoast() { description = "Dark Roast Coffee"; }
26     double cost() const override { return 0.99; }
27 };

```

2.7.2 Decorator base class and concrete condiments

CondimentDecorator inherits from Beverage and holds a `std::unique_ptr<Beverage>`. Concrete condiments override `getDescription()` and `cost()` to add their specific details and prices:

```

1 // Abstract Decorator
2 class CondimentDecorator : public Beverage {
3 protected:
4     std::unique_ptr<Beverage> wrapped;
5 public:
6     explicit CondimentDecorator(std::unique_ptr<Beverage> b)
7         : wrapped(std::move(b)) {}
8 };
9
10 // Concrete Decorator: Milk
11 class Milk : public CondimentDecorator {
12 public:
13     explicit Milk(std::unique_ptr<Beverage> b) : CondimentDecorator(std
14         ::move(b)) {}
15     std::string getDescription() const override {
16         return wrapped->getDescription() + ", Milk";
17     }
18     double cost() const override {
19         return wrapped->cost() + 0.10;
20     }
21 };
22
23 // Concrete Decorator: Mocha
24 class Mocha : public CondimentDecorator {
25 public:
26     explicit Mocha(std::unique_ptr<Beverage> b) : CondimentDecorator(std
27         ::move(b)) {}
28     std::string getDescription() const override {
29         return wrapped->getDescription() + ", Mocha";
30     }
31     double cost() const override {
32         return wrapped->cost() + 0.20;
33     }
34 };
35
36 // Concrete Decorator: Whip
37 class Whip : public CondimentDecorator {
38 public:
39     explicit Whip(std::unique_ptr<Beverage> b) : CondimentDecorator(std
40         ::move(b)) {}
41     std::string getDescription() const override {

```

```

39         return wrapped->getDescription() + ", Whip";
40     }
41     double cost() const override {
42         return wrapped->cost() + 0.15;
43     }
44 };

```

2.7.3 Client and dynamic wrapping

Client code constructs and decorates beverages dynamically at runtime:

```

1  int main() {
2      // Plain Espresso
3      std::unique_ptr<Beverage> drink1 = std::make_unique<Espresso>();
4      std::cout << drink1->getDescription() << " $" << drink1->cost() << "
5          \n";
6
7      // Dark Roast with double Mocha and Whip
8      std::unique_ptr<Beverage> drink2 = std::make_unique<DarkRoast>();
9      drink2 = std::make_unique<Mocha>(std::move(drink2));
10     drink2 = std::make_unique<Mocha>(std::move(drink2)); // Second
11     portion of Mocha
12     drink2 = std::make_unique<Whip>(std::move(drink2));
13
14     std::cout << drink2->getDescription() << " $" << drink2->cost() << "
15         \n";
16     return 0;
17 }

```

Compiling with `g++ -std=c++14 main.cpp` produces output where description strings and costs accumulate recursively through the decorator chain without modifying existing class definitions.

2.8 Technical considerations

2.8.1 Ownership and memory management

When decorating objects in C++, clear ownership boundaries are vital to avoid memory leaks or dangling pointers. Using `std::unique_ptr` establishes strict single ownership up the decorator chain: destroying the outermost decorator automatically recursively deletes all wrapped inner components. If decorators must be shared across threads or data structures, `std::shared_ptr` can be employed instead.

2.8.2 Order of execution dependence

Decorators execute their logic either before or after delegating to the wrapped component. If decorator actions are non-commutative (e.g., applying a 10% discount decorator vs. a \$1.00 surcharge decorator), the order in which decorators are wrapped directly dictates the final behavior and outcome.

2.8.3 Interface transparency vs. type inspection

Because decorators implement the component interface, clients interact with decorators as if they were base components. However, this transparency breaks down if the client relies on runtime type reflection or casting (e.g., `dynamic_cast<Espresso*>(drink.get())`). A wrapped `Espresso` object is contained within a `Mocha` decorator, so casting the decorator directly to `Espresso*` will return `nullptr`.

2.8.4 Performance and delegation overhead

Each added decorator introduces a layer of dynamic dispatch (virtual function calls) and pointer dereferencing. While modern CPU branch predictors handle standard delegation chains efficiently, deeply nested decorator chains (e.g., hundreds of wrappers) can accumulate minor stack and latency overhead compared to monolithic direct function calls.

2.8.5 Testing and mock decorators

Concrete decorators and concrete components can be unit-tested in complete isolation. Furthermore, test suites can inject mock decorators to verify that pre-processing (e.g., input verification) or post-processing (e.g., logging or price calculations) occurs as expected without instantiating actual production components.

2.9 Advantages and disadvantages

Table 2.1: Trade-offs of the Decorator pattern.

Advantages	Disadvantages
Provides greater flexibility than static inheritance.	Can result in a large number of small, visually similar classes.
Allows dynamic addition/removal of responsibilities at runtime.	Object instantiation becomes complex without Factory or Builder patterns.
Avoids feature-laden classes high up in the hierarchy.	Breaks object identity (<code>dynamic_cast</code> to concrete component fails).
Adheres strictly to the Open/Closed Principle (OCP).	Decorator ordering dependencies can cause subtle runtime bugs.
Supports multiple layers of identical responsibilities (e.g., double Mocha).	Deep delegation chains can complicate debugging and stack tracing.

2.10 Other real-world applications

Java I/O streams (`java.io`)

`BufferedInputStream` and `GZIPInputStream` decorate a base `InputStream` (such as `FileInputStream`), adding buffering and decompression:

```

1  // Java I/O Stream Decorator example concept:
2  // InputStream in = new GZIPInputStream(new BufferedInputStream(
    new FileInputStream("data.gz")));

```

GUI toolkits: window decoration

Graphical user interface frameworks decorate basic UI controls (such as a text panel) with visual embellishments like `ScrollbarDecorator` and `BorderDecorator` without modifying the base control's rendering logic.

Web development: HTTP middleware

Web application frameworks wrap core request handlers in middleware layers that add cross-cutting concerns like authentication checking, request logging, CORS headers, and response compression.

Graphics rendering pipelines

Render targets in game engines wrap base image buffers with post-processing filters (e.g., bloom, depth of field, color grading), applying dynamic visual effects sequentially before final display output.

2.11 Relationship to similar patterns

Decorator compared with related patterns

Pattern	Primary intent	Key distinction
Adapter	Convert one interface to match another expected interface.	Adapter changes the interface; Decorator preserves or enhances the interface.
Strategy	Alter an object's internal algorithm or strategy.	Strategy changes the guts of an object; Decorator changes the skin from the outside.
Proxy	Control access to an underlying subject.	Proxy manages lifecycle/access; Decorator adds functional responsibilities.
Composite	Treat individual objects and compositions uniformly.	Composite builds tree hierarchies; Decorator adds responsibilities along a single chain.
Facade	Provide a simplified entry point to a complex subsystem.	Facade defines a new simple interface; Decorator retains the exact component interface.

2.12 Multiple Choice Quiz

- Which category contains Decorator? (a) Creational (b) Structural (c) Behavioral (d) Concurrency
- What is its primary purpose? (a) Simplify subsystem access (b) Attach responsibilities to an object dynamically (c) Interchange algorithms at runtime (d) Ensure a single instance
- Why does Decorator implement the Component interface? (a) To share member variables (b) To remain transparent to clients by sharing the interface (c) To avoid virtual function calls (d) To prevent subclassing
- What happens when performing `dynamic_cast` on a decorated component to cast to a concrete component? (a) Memory leak (b) Compiler error (c) Returns `nullptr` because the outer decorator is not of that concrete type (d) Crashes the OS
- Which SOLID principle is most directly supported by Decorator? (a) Single Responsibility Principle (b) Open/Closed Principle (c) Liskov Substitution Principle (d) Interface Segregation Principle
- How does Decorator differ from Adapter? (a) Decorator changes the interface (b) Decorator preserves or enhances the component interface (c) Adapter uses composition while Decorator uses static methods (d) They are identical
- In modern C++, how is memory managed safely in a decorator chain? (a) Manual raw pointers (b) Cascading destruction with `std::unique_ptr` (c) Global singletons (d) Destructors are omitted

Answer key: 1(b), 2(b), 3(b), 4(c), 5(b), 6(b), 7(b).

Chapter 3

Facade Pattern

3.1 Real-world problem: operating a home theater

A home-theater system is simple from the viewer's perspective—start a movie and stop it later—but the operation requires coordination among several independent devices:

- the **amplifier** supplies sound and needs a source and volume;
- the **DVD player** powers on, plays, stops, and powers off;
- the **projector** powers on and selects the DVD input;
- the **screen** must be lowered before viewing and raised afterward;
- the **theater lights** must be dimmed and later restored.

Starting a movie requires the following ordered workflow:

1. dim the lights to 20 percent and lower the screen;
2. turn on the projector and select the DVD input;
3. turn on the DVD player;
4. turn on the amplifier, select DVD, and set the volume to 10; and
5. ask the DVD player to play the requested movie.

Ending the movie reverses the setup: stop playback, power off the amplifier, DVD player, and projector, raise the screen, and restore the lights. Forgetting an operation or changing the order can leave the system in an incorrect state.

3.2 Naive solution without Facade

The naive implementation creates every subsystem in the client and performs the entire workflow directly. The actual seminar source has the following structure:

```
1  int main() {
2      Amplifier amp;
3      DvdPlayer dvd;
4      Projector projector;
5      Screen screen;
6      TheaterLights lights;
7
8      // Client manually starts the movie.
9      lights.dim(20);
10     screen.down();
11     projector.on();
12     projector.setInput("DVD");
```

```
13     dvd.on();
14     amp.on();
15     amp.setSource("DVD");
16     amp.setVolume(10);
17     dvd.play("The Matrix");
18
19     // Client must also perform the complete shutdown sequence.
20     dvd.stop();
21     amp.off();
22     dvd.off();
23     projector.off();
24     screen.up();
25     lights.on();
26 }
```

This program produces the correct behavior, so the problem is not functional correctness. The problem is the structure of the client and the cost of future change.

3.3 Problems in the naive design

1. **Tight coupling.** The client depends on five concrete subsystem classes and must understand every relevant method.
2. **Complex orchestration.** Correct behavior depends on a long, ordered sequence of calls rather than one operation that expresses intent.
3. **Code duplication.** Every client that starts or stops a movie must repeat the workflow.
4. **Single Responsibility Principle violation.** The client manages its own application logic as well as home-theater coordination.
5. **Maintenance burden.** A changed device API or an additional component such as a subwoofer can affect many client locations.

3.4 Pattern intent and applicability

Facade is a **structural design pattern**. Its intent is to provide a unified interface to a set of interfaces in a subsystem and thereby define a higher-level interface that is easier to use [1]. It is appropriate when:

- clients need a small set of common workflows over a complex subsystem;
- dependencies between clients and implementation classes should be reduced; or
- a layered architecture needs a clear entry point into each layer.

Facade is a convenience boundary, not an access-control mechanism. Subsystem classes may remain public for advanced clients that need fine-grained control. The pattern therefore simplifies the normal path without forcing every possible operation into one interface. It defines new client-facing operations by combining existing subsystem capabilities; it does not add functionality to the subsystem itself.

3.5 General class structure

Figure 3.1 shows the general relationship. The client depends on a Facade, which delegates each high-level operation to the appropriate subsystem objects. An optional Additional Facade can expose a focused subset of operations when placing every workflow in one class would make the

primary Facade too broad. Clients and other facades may use this additional entry point. The subsystems do not need to know that either Facade exists.

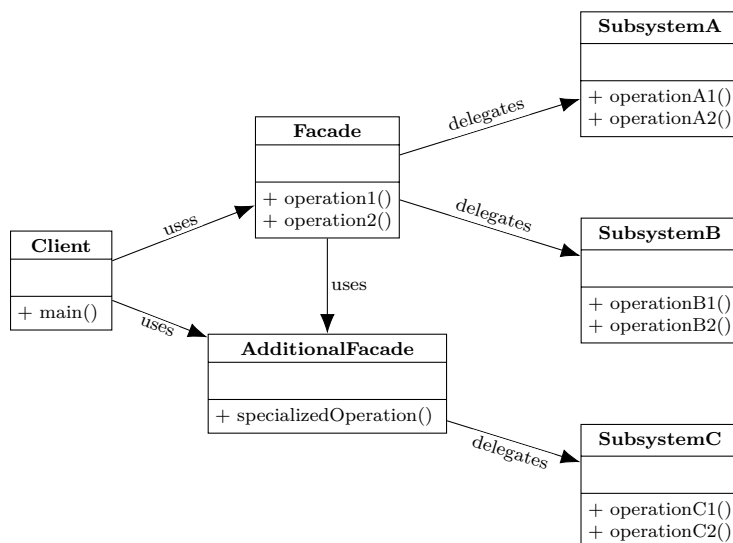


Figure 3.1: General Facade structure with an optional additional Facade.

The primary participants are:

- **Client**: requests a high-level service;
- **Facade**: knows which subsystem objects perform the request and coordinates them;
- **Additional Facade**: provides a smaller, cohesive entry point when the primary Facade would otherwise accumulate unrelated operations; and
- **Subsystem classes**: implement the actual domain operations and can still be used directly when necessary.

3.6 Home-theater class structure

In Figure 3.2, **HomeTheaterFacade** maintains associations to the five subsystem objects and coordinates their operations. Unlike Decorator, the Facade does not implement the same interface as those objects. It offers the high-level operations `watchMovie` and `endMovie`, each of which combines existing device capabilities rather than adding new functionality to the subsystem.

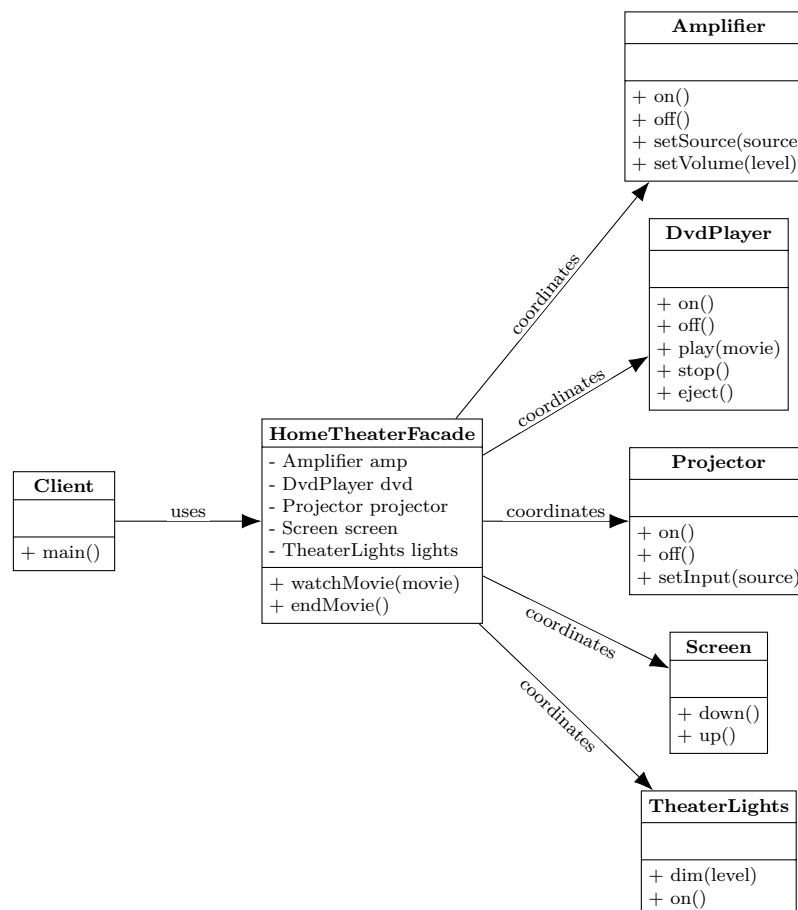


Figure 3.2: Facade structure for the home-theater problem.

3.7 Implementation with Facade

The seminar implementation uses header-only C++11 classes and the standard library. Subsystem objects are created by the client and injected into the Facade through its constructor:

```

1  class HomeTheaterFacade {
2  private:
3      Amplifier& amp;
4      DvdPlayer& dvd;
5      Projector& projector;
6      Screen& screen;
7      TheaterLights& lights;
8
9  public:
10     HomeTheaterFacade(Amplifier& a, DvdPlayer& d, Projector& p,
11                       Screen& s, TheaterLights& l)
12         : amp(a), dvd(d), projector(p), screen(s), lights(l) {}
13
14     void watchMovie(const std::string& movie) {
15         lights.dim(20);
16         screen.down();
17         projector.on();
18         projector.setInput("DVD");
19         dvd.on();
20         amp.on();
  
```

```
21     amp.setSource("DVD");
22     amp.setVolume(10);
23     dvd.play(movie);
24 }
25
26 void endMovie() {
27     dvd.stop();
28     amp.off();
29     dvd.off();
30     projector.off();
31     screen.up();
32     lights.on();
33 }
34 };
```

References are suitable for this demonstration for three reasons. They avoid copying objects that conceptually represent hardware, guarantee non-null dependencies, and communicate that the Facade uses but does not own the subsystems. Consequently, the subsystem objects must outlive the Facade.

Client code now communicates intent through two calls:

```
1  #include "HomeTheaterFacade.h"
2
3  int main() {
4      Amplifier amp;
5      DvdPlayer dvd;
6      Projector projector;
7      Screen screen;
8      TheaterLights lights;
9
10     HomeTheaterFacade theater(amp, dvd, projector, screen, lights);
11     theater.watchMovie("The Matrix");
12     std::cout << "\n--- Movie is playing ---\n\n";
13     theater.endMovie();
14 }
```

Both the naive and Facade programs compile with `g++ -std=c++11` and produce the same sequence of device output. The improvement is therefore internal design quality: orchestration is defined once and ordinary clients no longer depend on its details.

3.8 Technical considerations

3.8.1 Configuration and flexibility

The demonstration hardcodes a 20-percent light level, volume 10, and DVD input to keep the workflow visible. A production Facade could accept a configuration object while retaining convenient defaults. Advanced clients can also bypass the Facade and call `amp.setVolume` directly with a custom value.

3.8.2 Exception safety

If a device operation fails midway through `watchMovie`, earlier devices may remain active. A production implementation should provide rollback or RAII-based cleanup and must decide what

to do if cleanup itself fails. This is a workflow-transaction concern that the Facade conveniently centralizes.

3.8.3 Open/Closed Principle trade-off

Adding a `Subwoofer` will usually modify the Facade rather than every workflow client, which is a substantial maintenance improvement. However, the current constructor-injection design may also require a change at the application's dependency-assembly point to supply the new object. Strict OCP requirements can be addressed with subsystem interfaces, grouped dependencies, or a specialized Facade, but such flexibility should be justified by expected change rather than added automatically.

3.8.4 Avoiding a god object

A Facade should primarily delegate and coordinate a cohesive set of use cases. It should not absorb unrelated domain state and business logic. When its scope grows, focused interfaces such as `VideoPlaybackFacade` and `AudioCalibrationFacade` are preferable to one universal Facade.

3.9 Advantages and disadvantages

Table 3.1: Trade-offs of the Facade pattern.

Advantages	Disadvantages
Isolates ordinary clients from subsystem complexity.	A broad Facade can become a god object.
Reduces coupling and duplicated orchestration.	Adds another delegation layer.
Improves readability through use-case-oriented methods.	A simplified API may omit advanced options.
Localizes common workflow changes.	Changes to injected dependencies may affect the composition root.
Preserves direct subsystem access when needed.	Clients can bypass the Facade and reintroduce coupling.

3.10 Other real-world applications

Web development: API Gateway

A mobile or web client makes one request to a gateway, which handles authentication and coordinates catalog, inventory, ordering, payment, and notification services before returning a unified response.

Mobile development: hardware abstraction

A method such as `Camera.takePhoto` can coordinate sensor access, buffers, image processing, device drivers, and file output behind one operation.

Game development: engine API

High-level operations such as playing a sound or applying force shield game code from rendering, audio, physics, input, and hardware-acceleration details.

Database access: JDBC and ORMs

Operations such as `session.save` hide connection management, SQL generation, database dialects, network protocols, and result processing.

3.11 Relationship to similar patterns

Facade compared with related patterns

Pattern	Primary intent	Key distinction
Adapter	Make incompatible interfaces work together.	Usually translates one object to an existing target interface.
Mediator	Coordinate communication among peer objects.	Colleagues communicate through the mediator; Facade communication is normally client-to-subsystem.
Decorator	Add behavior while preserving an interface.	Decorators share the component interface; a Facade creates a simpler interface.
Proxy	Control access to another object.	A Proxy commonly preserves the subject interface and may enforce access or lifecycle rules.
Singleton	Ensure one instance and a global access point.	A Facade may be a Singleton, but global state harms testing and is not required by Facade.

3.12 Multiple Choice Quiz

1. Which category contains Facade? (a) Creational (b) Structural (c) Behavioral (d) Concurrency
2. What is its primary purpose? (a) Object creation (b) A simplified subsystem interface (c) One-to-many notification (d) Tree composition
3. In the home-theater example, what changes for an ordinary client? (a) Devices disappear (b) It calls the Facade instead of orchestrating every subsystem (c) Volume becomes automatic (d) Order no longer matters
4. What is a principal risk? (a) Increased inheritance (b) A god-object Facade (c) Forced subsystem rewriting (d) No direct access
5. Which is not a typical Facade example? (a) API Gateway (b) Singleton logger (c) ORM (d) Game-engine API
6. Does Facade prohibit direct subsystem access? (a) Always (b) No (c) Only in C++ (d) Only for private methods
7. Where should a common new subsystem workflow normally be coordinated? (a) Every client (b) The Facade (c) The compiler (d) A decorator

Answer key: 1(b), 2(b), 3(b), 4(b), 5(b), 6(b), 7(b).

Chapter 4

Strategy Pattern

4.1 Real-world problem: navigation route planning

A navigation application must calculate routes between an origin and a destination for several transport modes. Although every mode has the same goal, each requires a distinct algorithm and different data:

- **Driving** prioritizes highways, travel time, traffic conditions, and toll roads.
- **Walking** prioritizes sidewalks, pedestrian crossings, footbridges, and park shortcuts while avoiding roads unsafe for pedestrians.
- **Public transit** evaluates bus timetables, metro lines, walking connections, and transfers.
- **Bicycling** prioritizes bicycle lanes, safe streets, and favorable elevation profiles.

The set of algorithms is expected to grow as the application adds electric scooters, sightseeing routes, accessibility-aware routing, or eco-friendly driving. The design challenge is to support these variations and switch between them at runtime without concentrating every algorithm in one class.

4.1.1 Real-world analogy

Consider a traveler who needs to reach an airport. The destination remains the same, but the traveler can take a bus, hire a taxi, ride a bicycle, or drive a car. Each option is a transportation strategy that makes different trade-offs: the bus may minimize cost, a taxi may prioritize convenience and travel time, a bicycle may suit a short trip with little luggage, and a car may offer greater flexibility. The traveler selects an option according to factors such as budget, time, distance, weather, and luggage. Changing the selected option changes how the journey is performed without changing its overall goal, illustrating how interchangeable strategies provide alternative ways to complete the same task [3].

4.2 Naive solution without Strategy

The naive seminar implementation stores a `TransportMode` value in `Navigator`. Its `buildRoute` method selects an algorithm with a conditional chain:

```
1 enum class TransportMode {  
2     DRIVING,  
3     WALKING,  
4     PUBLIC_TRANSIT
```



```

5 };
6
7 class Navigator {
8 private:
9     TransportMode mode;
10
11 public:
12     Navigator(TransportMode initial = TransportMode::DRIVING)
13         : mode(initial) {}
14
15     void setMode(TransportMode selected) {
16         mode = selected;
17     }
18
19     void buildRoute(const std::string& origin,
20                    const std::string& destination) {
21         std::cout << "Calculating route from " << origin
22                   << " to " << destination << "...\\n";
23
24         if (mode == TransportMode::DRIVING) {
25             std::cout << "Finding the fastest highway route.\\n";
26         } else if (mode == TransportMode::WALKING) {
27             std::cout << "Finding pedestrian paths and shortcuts.\\n";
28         } else if (mode == TransportMode::PUBLIC_TRANSIT) {
29             std::cout << "Checking bus and metro schedules.\\n";
30         } else {
31             std::cout << "Error: unknown transport mode.\\n";
32         }
33     }
34 };

```

The client changes the enumeration before each request:

```

1 Navigator navigator(TransportMode::DRIVING);
2 navigator.buildRoute("Home", "Downtown Office");
3
4 navigator.setMode(TransportMode::WALKING);
5 navigator.buildRoute("Downtown Office", "City Park");
6
7 navigator.setMode(TransportMode::PUBLIC_TRANSIT);
8 navigator.buildRoute("City Park", "Home");

```

This design works for three simple modes, but every routing rule remains part of one monolithic class.

4.3 Problems in the naive design

1. **Open/Closed Principle violation.** Adding BICYCLING requires editing the enumeration and the conditional logic in Navigator.
2. **Single Responsibility Principle violation.** The context manages route requests and also implements every routing algorithm.
3. **Monolithic growth.** Traffic handling, timetable parsing, elevation analysis, and future route types accumulate in one method or class.
4. **Poor isolated testing.** A routing algorithm cannot be tested as an independent unit; tests must configure and execute the complete Navigator.

5. **Limited runtime composition.** Behavior is selected from a fixed enumeration rather than supplied as a configurable algorithm object.
6. **Merge and maintenance risk.** Multiple developers changing unrelated algorithms must edit the same central file.

4.4 Pattern intent and applicability

Strategy is a **behavioral design pattern**. Its intent is to define a family of algorithms, encapsulate each one, and make them interchangeable so that an algorithm can vary independently from the clients that use it [1].

The pattern is appropriate when:

- a system offers several meaningful variants of one algorithm;
- a class contains large conditional branches that select behavior;
- algorithms should be replaced or configured at runtime;
- algorithm-specific data and dependencies should be isolated; or
- each algorithm needs independent development and testing.

Strategy applies the principle *favor composition over inheritance*. The Navigator context stores a strategy object and delegates routing to it. Unlike a design with a separate Navigator subclass for each mode, one context object can change its behavior without being destroyed and recreated.

4.5 General class structure

Figure 4.1 shows the general participants and dependency direction.

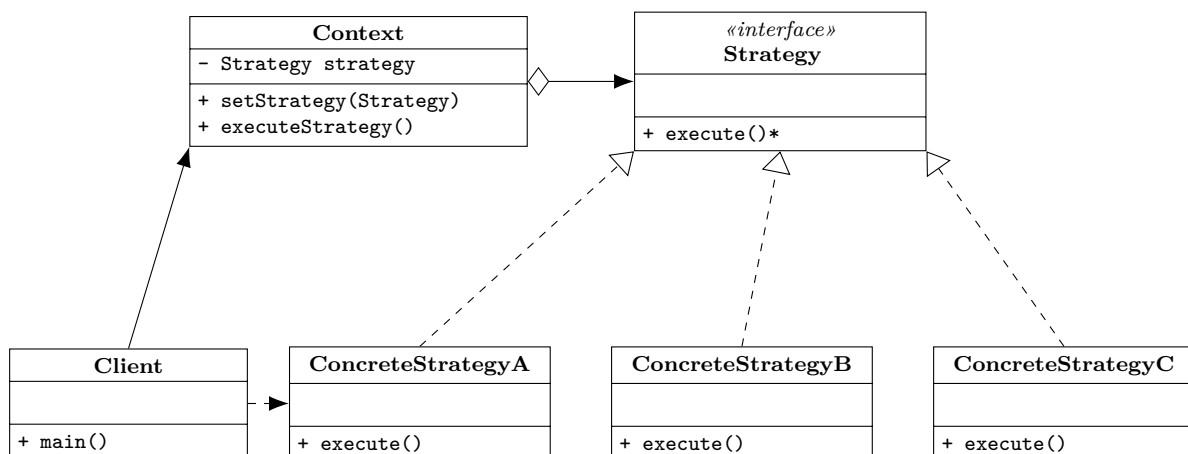


Figure 4.1: General Strategy pattern structure.

- The **Strategy** interface declares the operation shared by all algorithms.
- Each **Concrete Strategy** implements one algorithm.
- The **Context** stores a Strategy reference and delegates the operation instead of implementing the algorithm.
- The **Client** selects a concrete strategy, configures the Context, and may replace the strategy while the program is running.

The Context depends only on the abstraction. Concrete strategies are independent of one another and do not need to know which other algorithms are available.

4.6 Navigation class structure

Figure 4.2 maps the general roles to the seminar example. Navigator is the Context, RouteStrategy is the common interface, and the four transport-mode classes are concrete strategies.

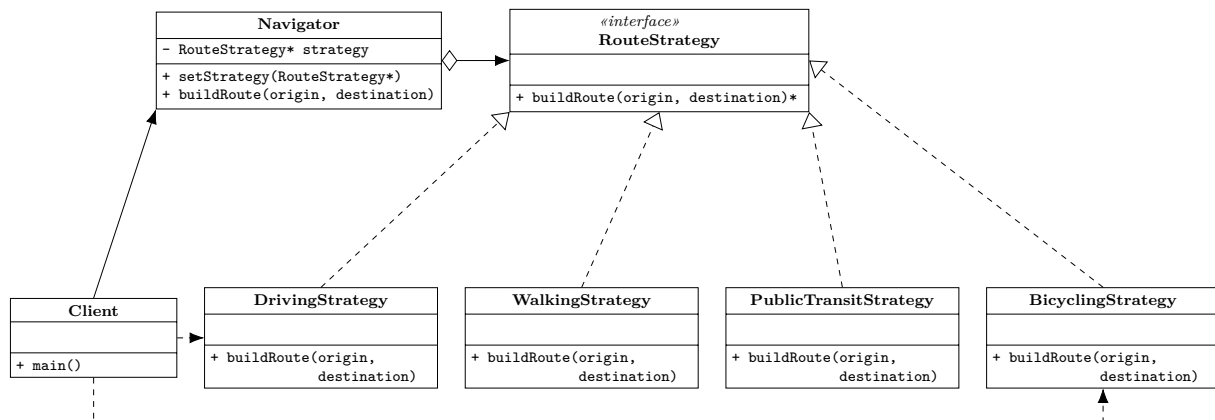


Figure 4.2: Strategy structure for the navigation application.

The hollow-diamond association expresses that Navigator delegates to a strategy object. The demonstration uses a non-owning pointer, so the Client creates both the Context and concrete strategies.

4.7 Implementation with Strategy

4.7.1 Strategy interface and concrete algorithms

The common interface declares a pure virtual route-building operation and a virtual destructor:

```

1 class RouteStrategy {
2 public:
3     virtual ~RouteStrategy() = default;
4
5     virtual void buildRoute(const std::string& origin,
6                             const std::string& destination) const = 0;
7 };
  
```

The virtual destructor makes destruction through a base-class pointer safe. The concrete classes implement the same operation with mode-specific behavior:

```

1 class DrivingStrategy : public RouteStrategy {
2 public:
3     void buildRoute(const std::string& origin,
4                     const std::string& destination) const override {
5         std::cout << "Mode: DRIVING\n";
6         std::cout << "Highway priority, live traffic, and toll roads.\n"
7         ;
8         std::cout << "Estimated time: 25 mins (18.5 km)\n";
9     }
10 };
11
12 class BicyclingStrategy : public RouteStrategy {
13 public:
  
```

```

13     void buildRoute(const std::string& origin,
14                    const std::string& destination) const override {
15         std::cout << "Mode: BICYCLING\n";
16         std::cout << "Bike lanes and elevation-profile optimization.\n";
17         std::cout << "Estimated time: 50 mins (17.1 km)\n";
18     }
19 };

```

The project also supplies walking and public-transit implementations. Adding the new bicycling implementation does not require a change to Navigator or the other strategy classes.

4.7.2 Context

The Context stores the currently selected strategy and forwards route requests:

```

1  class Navigator {
2  private:
3      const RouteStrategy* strategy;
4
5  public:
6      explicit Navigator(const RouteStrategy* initial = nullptr)
7          : strategy(initial) {}
8
9      void setStrategy(const RouteStrategy* selected) {
10         strategy = selected;
11     }
12
13     void buildRoute(const std::string& origin,
14                    const std::string& destination) const {
15         std::cout << "Calculating route from " << origin
16                 << " to " << destination << "... \n";
17
18         if (strategy) {
19             strategy->buildRoute(origin, destination);
20         } else {
21             std::cout << "Error: no routing strategy set.\n";
22         }
23     }
24 };

```

The null check prevents dereferencing an unset pointer. A production design could instead require a valid strategy in the constructor or inject a Null Object strategy that performs defined fallback behavior.

4.7.3 Client and runtime switching

The actual client creates four stack-allocated strategy objects and switches the Context between them:

```

1  DrivingStrategy driving;
2  WalkingStrategy walking;
3  PublicTransitStrategy transit;
4  BicyclingStrategy bicycling;
5
6  Navigator navigator(&driving);
7  navigator.buildRoute("Home", "Downtown Office");

```

```
8
9 navigator.setStrategy(&walking);
10 navigator.buildRoute("Downtown Office", "City Park");
11
12 navigator.setStrategy(&transit);
13 navigator.buildRoute("City Park", "Home");
14
15 navigator.setStrategy(&bicycling);
16 navigator.buildRoute("Home", "Riverside Bike Trail");
```

The implementation compiles as C++11 and demonstrates runtime polymorphism. The selected object must outlive the Navigator because the pointer is non-owning.

4.8 Technical considerations

4.8.1 Pointers, ownership, and object slicing

Passing a concrete strategy by pointer or reference preserves virtual dispatch. Passing it by value as a `RouteStrategy` would be invalid because the interface is abstract; more generally, passing polymorphic derived objects by base value causes object slicing. If Navigator should own its strategy, `std::unique_ptr<RouteStrategy>` is a clearer alternative. Shared ownership should be used only when the lifetime genuinely has multiple owners.

4.8.2 Const correctness and demonstration parameters

The declaration `const RouteStrategy*` makes the pointed-to strategy const through this pointer, while the pointer itself remains mutable so that `setStrategy` can replace it. The `buildRoute` member functions are const-qualified, indicating that route calculation does not mutate the strategy object. In the educational source, the concrete strategies receive `origin` and `destination`, but only the Context prints them. A production algorithm would consume these parameters; otherwise their names can be omitted in the definition to avoid unused-parameter compiler warnings.

4.8.3 Client coupling and creation

The Client still needs to select a concrete algorithm. This coupling is localized to configuration rather than spread through the Context and routing workflow. A Factory or dependency-injection configuration can further separate strategy creation from the client that executes routes.

4.8.4 Performance

Virtual dispatch adds one small indirect call and can inhibit inlining. This cost is negligible compared with route search, network access, payment processing, or other substantial algorithms. Highly performance-sensitive inner loops may instead use templates, the Curiously Recurring Template Pattern, or `std::variant` for static or closed-set polymorphism.

4.8.5 Testing

Each concrete strategy can be unit-tested independently. Navigator can be tested with a mock `RouteStrategy` that records the supplied origin and destination, proving that the Context delegates correctly without invoking a real routing service.

4.9 Advantages and disadvantages

Table 4.1: Trade-offs of the Strategy pattern.

Advantages	Disadvantages
Adds algorithms without modifying the Context.	Introduces more classes and objects.
Separates algorithm logic from context logic.	Clients must understand enough to select a strategy.
Supports runtime algorithm replacement.	Adds configuration and virtual-call indirection.
Removes large behavior-selection conditionals.	Can over-engineer simple, stable behavior.
Enables focused unit tests and mocks.	Strategy objects may require careful lifetime management.

4.10 Other real-world applications

Web development: authentication and payment

Authentication middleware can select local-password, Google OAuth, or another provider strategy. Checkout systems can delegate payment to credit-card, PayPal, Apple Pay, or other payment strategies.

Mobile development: layout and caching

Collection views can use linear, grid, or staggered layout strategies. Image libraries can switch among memory-cache, disk-cache, and network-only policies.

Game development: artificial intelligence and physics

A non-player character can replace patrol, attack, and flee strategies as its situation changes. Physics engines can choose collision-detection algorithms according to scene density.

Data processing: sorting and compression

Software can select a sorting, compression, or serialization algorithm based on data characteristics, performance requirements, or output format.

4.11 Relationship to similar patterns

Strategy compared with related patterns

Pattern	Primary intent	Key distinction
State	Represent behavior associated with internal state.	State transitions are often initiated internally; strategies are normally selected externally.
Template Method	Vary steps in an inherited algorithm skeleton.	Template Method uses class inheritance; Strategy uses object composition and runtime delegation.
Command	Encapsulate a request for execution, queuing, logging, or undo.	Strategy represents how an algorithm is performed rather than a request to perform an action.
Decorator	Add responsibilities while preserving an object's interface.	Decorator wraps to add behavior; Strategy replaces the delegated algorithm.
Facade	Simplify access to a complex subsystem.	Facade offers a higher-level interface; Strategy exposes interchangeable implementations of one operation.

4.12 Multiple Choice Quiz

1. Which category contains Strategy? (a) Creational (b) Structural (c) Behavioral (d) Architectural
2. What is its primary intent? (a) Adapt interfaces (b) Encapsulate interchangeable algorithms (c) Ensure one instance (d) Simplify a subsystem
3. Which principle is most directly supported when a new strategy is added without changing Navigator? (a) Liskov Substitution (b) Open/Closed (c) Interface Segregation (d) Law of Demeter
4. What is the Context's role? (a) Implement every algorithm (b) Store a Strategy and delegate to it (c) Prevent runtime changes (d) Create JSON
5. What is a possible disadvantage? (a) Strategies cannot be tested (b) Clients must select among strategies (c) Context must change for every strategy (d) Runtime swapping is impossible
6. How does Strategy differ from Template Method? (a) Strategy uses composition; Template Method uses inheritance (b) Strategy uses inheritance; Template Method uses composition (c) Both require a Singleton (d) They have identical intent
7. Why use a pointer or reference to the strategy in C++? (a) To force inlining (b) To preserve polymorphism and avoid slicing (c) To create a Singleton (d) To prevent testing

Answer key: 1(c), 2(b), 3(b), 4(b), 5(b), 6(a), 7(b).

Chapter 5

Comparison and Combined Use

Table 5.1: Comparison of the three seminar patterns.

	Decorator	Strategy	Facade
Category	Structural	Behavioral	Structural
Main question	What behavior can be added?	Which algorithm should run?	How can this subsystem be simpler?
Mechanism	Recursive wrapping	Delegation to a policy object	Delegation to subsystem objects
Interface	Same as wrapped component	Strategy-specific interface	New, simplified interface
Runtime change	Add/remove wrapper layers	Replace the strategy	Usually use fixed workflows
Primary benefit	Flexible extension	Interchangeable algorithms	Reduced client coupling
Typical risk	Too many wrapper layers	Too many strategy classes	God-object facade

The patterns can cooperate in one system. For example, a route-planning service may expose a **Facade** for simple trip planning, use a **Strategy** to select driving or walking algorithms, and apply **Decorators** for caching, metrics, or logging. Pattern choice should follow the design problem; patterns are not goals by themselves.

Chapter 6

Conclusion

Decorator, Strategy, and Facade show how object composition can improve change management in different ways. Decorator extends individual objects without a large inheritance tree. Strategy separates variable algorithms from the context that uses them. Facade shields clients from subsystem complexity through a cohesive high-level interface. Their benefits are strongest when the relevant variation or complexity is genuine; applying them to trivial code can increase indirection without sufficient return. A good design therefore balances flexibility, clarity, ownership, testability, and expected future change.

Appendix A

Submission Checklist

The seminar requirements specify the following submission artifacts:

- AI Usage Declaration and AI Chat Log;
- **Changes.md**, recording changes since the last presentation;
- the presentation in Markdown, PPTX, or Canvas format;
- the presentation in PDF format;
- the report in Word, $\text{\LaTeX}$, or Markdown format, including references;
- source code; and
- the seminar member-contribution evaluation.

Appendix B

AI Usage Declaration

B.1 Declaration of AI use

Group 53 declares that generative AI was used as a support tool during the preparation of this seminar report. AI assistance was disclosed rather than treated as an academic source, and the group remains responsible for the accuracy, understanding, and oral defense of every submitted part.

B.2 AI Usage Notes

Required detail	Disclosure
Tool, model, and platform	OpenAI Codex / Google Gemini, accessed through the Prism $\text{\LaTeX}$ editor.
Access period	August 28, 2026 – August 29, 2026.
Purposes of use	Organizing the report; drafting and revising explanations; formatting C++ examples and comparison tables; drawing TikZ UML diagrams.

B.3 Student responsibility statement

By submitting this report, both members confirm that they have reviewed the final text and code, can explain the AI-assisted material, and have verified factual claims using the project files and references listed in this report.

Bibliography

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] E. Freeman and E. Robson, *Head First Design Patterns*, 2nd ed. O'Reilly Media, 2020.
- [3] Refactoring.Guru, “Strategy Design Pattern”.
<https://refactoring.guru/design-patterns/strategy>
- [4] SourceMaking, “Design Patterns”.
https://sourcemaking.com/design_patterns
- [5] ISO/IEC, *ISO International Standard ISO/IEC 14882:2020 — Programming Languages — C++*, 2020.